

Engenharia de Software Conceitos



Profa. Patricia Rucker de Bassi

Conceitos

- "Engenharia de Software é o estabelecimento e o emprego de sólidos **princípios de engenharia** de modo a obter software de maneira **econômica**, que seja confiável e funcione de forma **eficiente** em máquinas reais". (Fritz Bauer)
- "A engenharia de software é uma disciplina da engenharia que se ocupa de todos os aspectos da **produção de software**, desde os estágios iniciais de **especificação** do sistema até a **manutenção** desse sistema, depois que ele entrou em operação. (Ian Sommerville)

Conceitos

- “A Engenharia de Software baseia-se no estudo e aplicação de uma **abordagem sistemática**, disciplinada e quantificável para o desenvolvimento, operação e manutenção de software.” (IEEE Instituto de Engenheiros Eletricistas e Eletrônicos)
- “A Engenharia de Software é uma tecnologia em 3 camadas: **processos, métodos e ferramentas**. E a base de todas essas camadas é o foco na qualidade do software desenvolvido. O trabalho associado à engenharia de software pode ser categorizado em 3 fases genéricas, independentemente da área de aplicação, do tamanho do projeto ou de sua complexidade: **definição, desenvolvimento e manutenção**.” (Roger Pressman)

Engenharia de Software - ES

- É uma área da computação voltada à **especificação, desenvolvimento, manutenção e criação** de sistemas de *software*, com aplicação de tecnologias e práticas de **gerência de projetos** e outras disciplinas, **visando organização, produtividade e qualidade**.
- Tecnologias e práticas
 - linguagens de programação, banco de dados, ferramentas, plataformas, bibliotecas, padrões, processos e qualidade de software.
- Fundamentos científicos
 - modelos abstratos e precisos que permitem ao engenheiro especificar, projetar, implementar e manter sistemas de software, avaliando e garantindo sua qualidade.
- Mecanismos
 - planejar e gerenciar o processo de desenvolvimento de um sistema computacional.

"Software está se tornando mais e mais uma parte integrante da sociedade e, se, por alguma razão, algum dia todo o software parasse de funcionar, o efeito no mundo ocidental seria catastrófico. Embora as pessoas que trabalham em disciplinas técnicas estejam cientes deste fato há muitos anos, não era claro para o público em geral, até que o problema do ano 2000 e suas consequências tivessem sido discutidos abertamente na mídia."

[Bosch, 2000]

Bug do milênio



Bug do milênio

- O Bug do Milênio foi um acontecimento que ocorreu no fim do século XX, e passou de um simples problema relacionado à informática para a preocupação de todo o mundo.
- DD/MM/AA ao invés de DD/MM/AAAA
- 31/12/99 → 01/01/00
- Cobol



Origem da Engenharia de Software

- *NATO Software Engineering Conference (1968)*



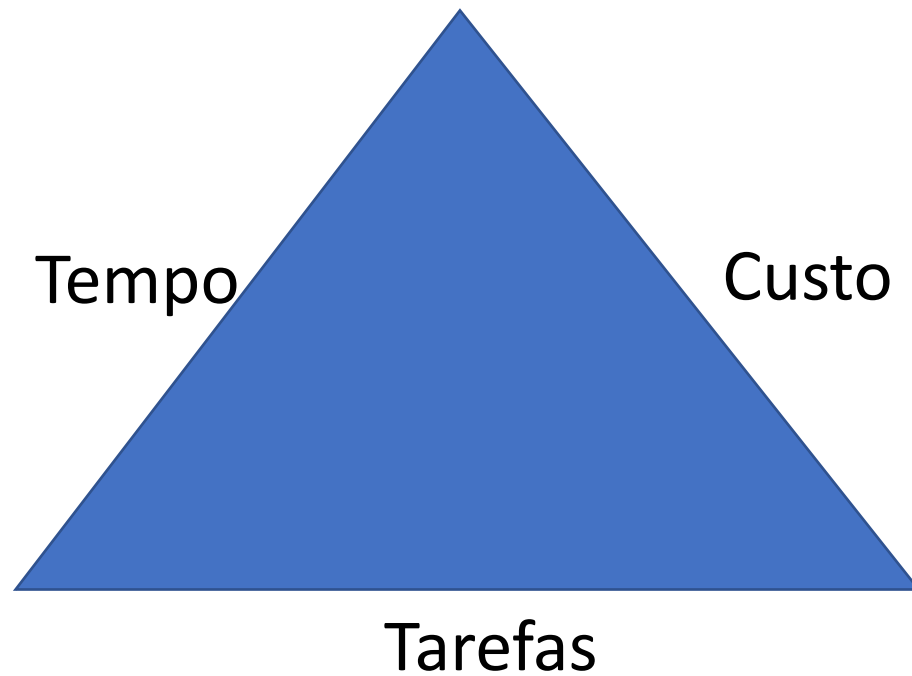
Garmish, Alemanha, 1968

- Crise do software
 - Dificuldade em desenvolver software complexos de forma segura e previsível
 - Dificuldade em cumprir os cronogramas e especificações de grandes projetos de software
- Termo Engenharia de Software
 - Necessidade de trazer mais “engenharia para o software”(McIlroy, 1968)

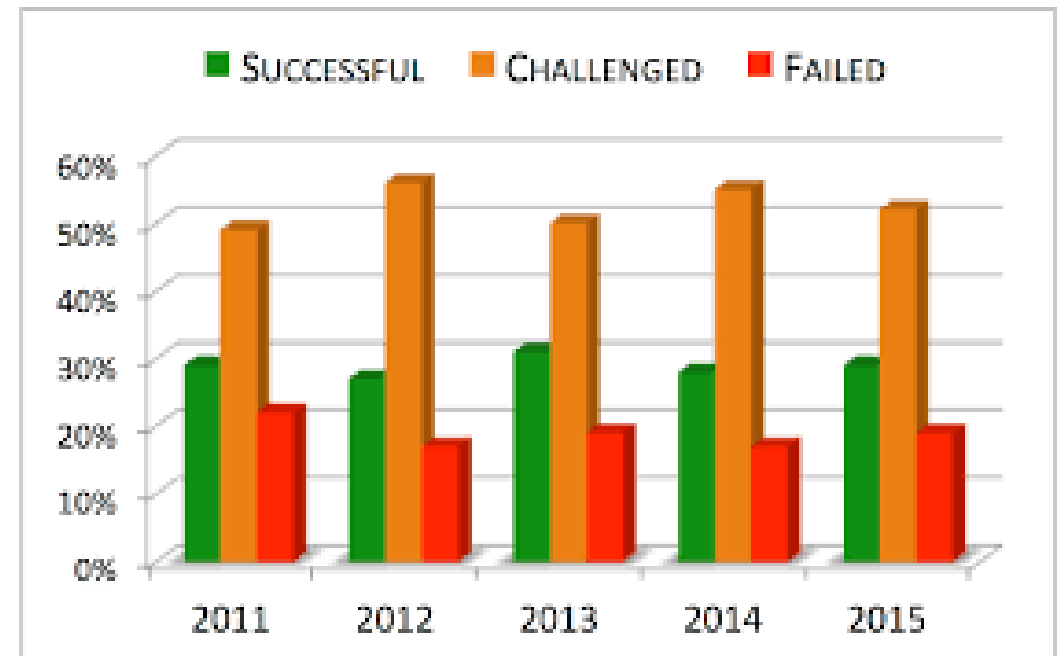
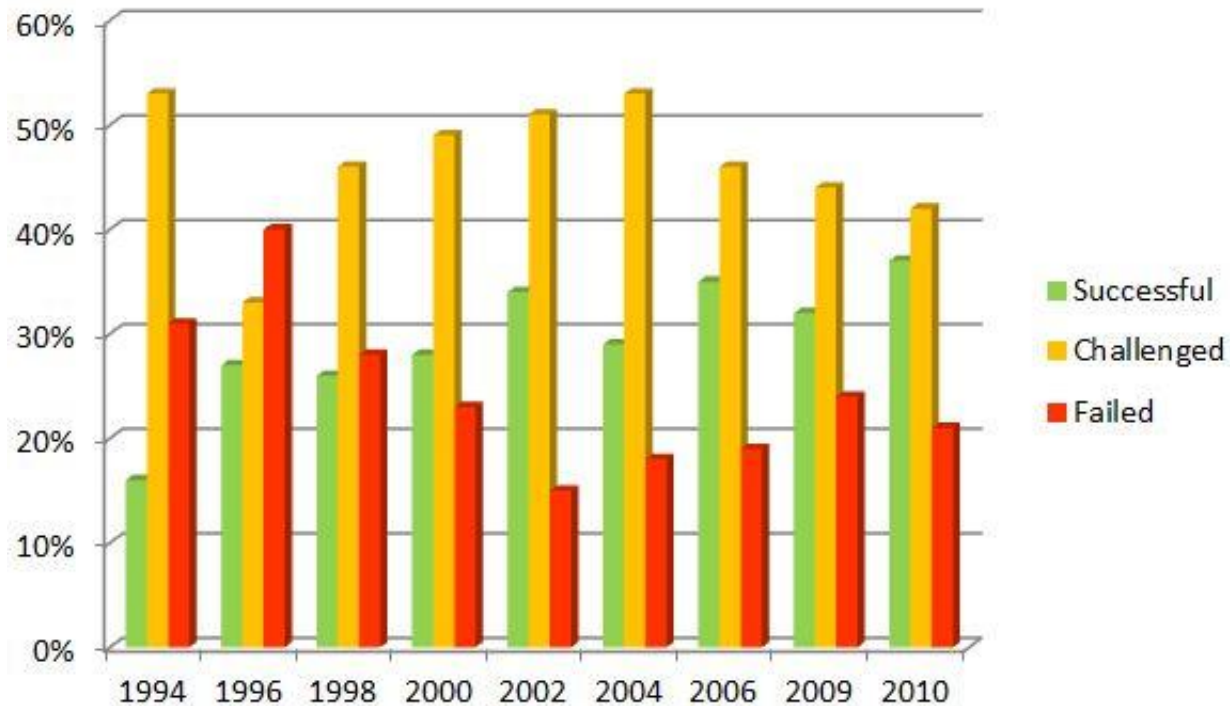
Problemas

- Software ainda é desenvolvido “sob medida” apesar de se falar muito em componentes, ao contrário do hardware – que é uma composição de produtos de prateleira.
- Em geral, os projetos de software:
 - Ultrapassam o orçamento
 - Atrasam o cronograma
 - São cancelados
 - Não atendem os requisitos dos clientes/usuários
 - Deixam o usuário extremamente insatisfeitos
- <https://teamstage.io/project-management-statistics/#:~:text=1.,rate%20to%2020%25%20or%20below.>

Custo X Tempo (Prazo) X Tarefas(escopo)



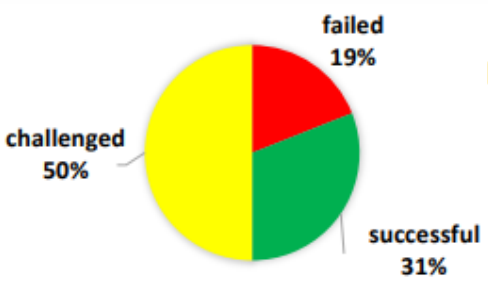
Sucessos em projetos de TI



Sucessos em Projetos de TI - 2020

Project Success Quick Reference Card

Based on CHAOS 2020: Beyond Infinity Overview. January 2021, QRC by Henny Portman



Modern measurement (software projects)

Good Sponsor, Good Team, and Good Place are the only things we need to improve and build on to improve project performance.



The Good Place is where the sponsor and team work to create the product. It's made up of the people who support both sponsor and team. These people can be helpful or destructive. It's imperative that the organization work to improve their skills if a project is to succeed. This area is the hardest to mitigate, since each project is touched by so many people. Principles for a Good Place are:

- The Decision Latency Principle
- The Emotional Maturity Principle
- The Communication Principle
- The User Involvement Principle
- The Five Deadly Sins Principle
- The Negotiation Principle
- The Competency Principle
- The Optimization Principle
- The Rapid Execution Principle
- The Enterprise Architecture Principle



Successful project Resolution by Good Place Maturity Level:

highly mature	50%
mature	34%
moderately mature	23%
not mature	23%

The Good Team is the project's workhorse. They do the heavy lifting. The sponsor breathes life into the project, but the team takes that breath and uses it to create a viable product that the organization can use and from which it derives value. Since we recommend small teams, this is the second easiest area to improve. Principles for a Good Team are:

- The Influential Principle
- The Mindfulness Principle
- The Five Deadly Sins Principle
- The Problem-Solver Principle
- The Communication Principle
- The Acceptance Principle
- The Respectfulness Principle
- The Confrontationist Principle
- The Civility Principle
- The Driven Principle



Successful project Resolution by Good Team Maturity Level:

highly mature	66%
mature	46%
moderately mature	21%
not mature	1%

The Good Sponsor is the soul of the project. The sponsor breathes life into a project, and without the sponsor there is no project. Improving the skills of the project sponsor is the number-one factor of success – and also the easiest to improve upon, since each project has only one. Principles for a Good Sponsor are:

- The Decision Latency principle
- The Vision Principle
- The Work Smart Principle
- The Daydream Principle
- The Influence Principle
- The Passionate Principle
- The People Principle
- The Tension Principle
- The Torque Principle
- The Progress Principle



Successful project Resolution by Good Sponsor Maturity Level:

highly mature	67%
mature	33%
moderately mature	21%
not mature	18%

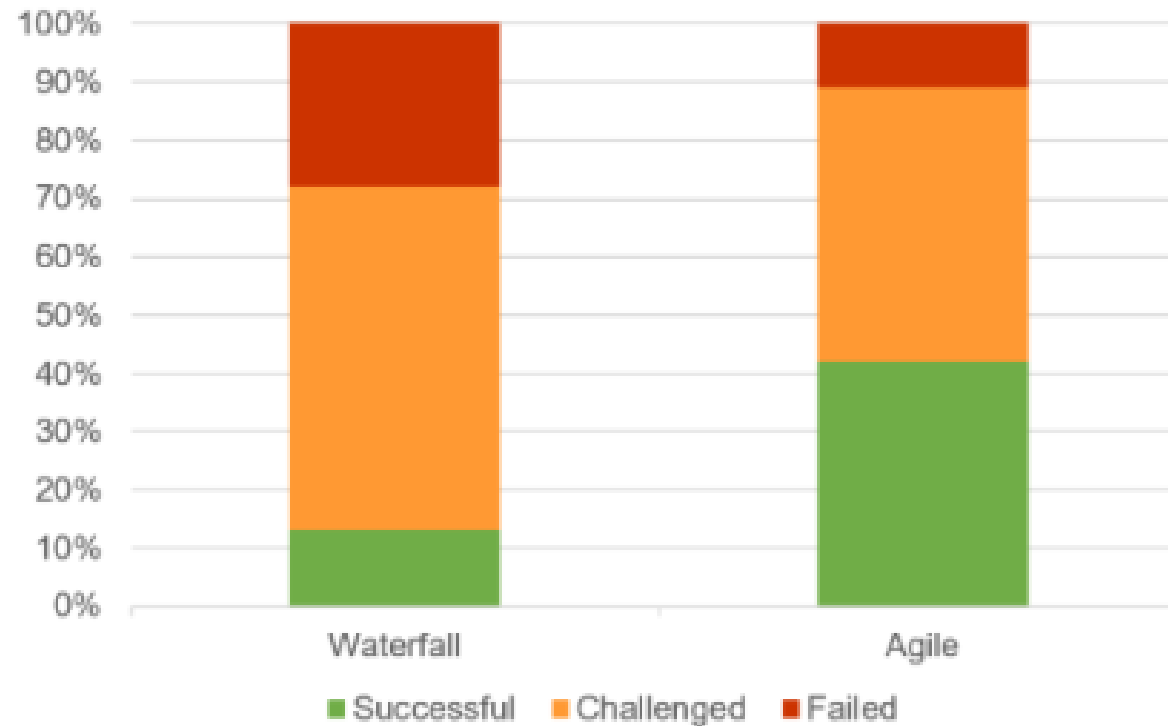
Sucessos em projetos de TI

CHAOS RESOLUTION BY PROJECT SIZE

	SUCCESSFUL	CHALLENGED	FAILED
Grand	2%	7%	17%
Large	6%	17%	24%
Medium	9%	26%	31%
Moderate	21%	32%	17%
Small	62%	16%	11%
TOTAL	100%	100%	100%

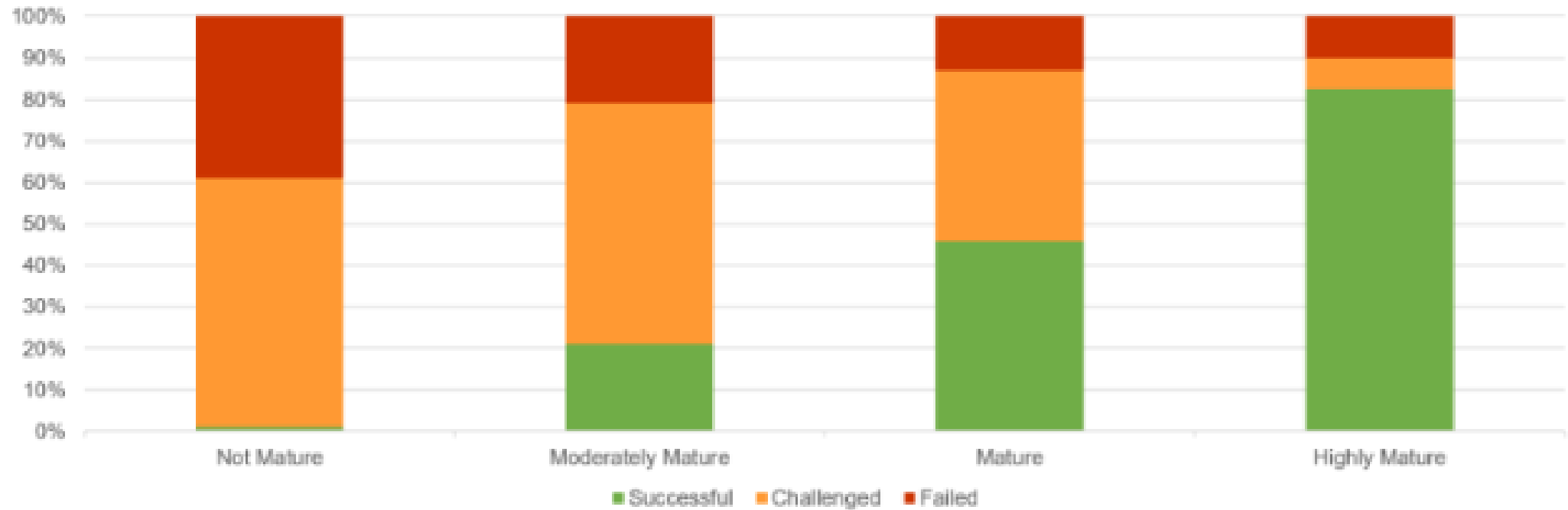
The resolution of all software projects by size from FY2011-2015 within the new CHAOS database.

Sucessos em projetos de TI



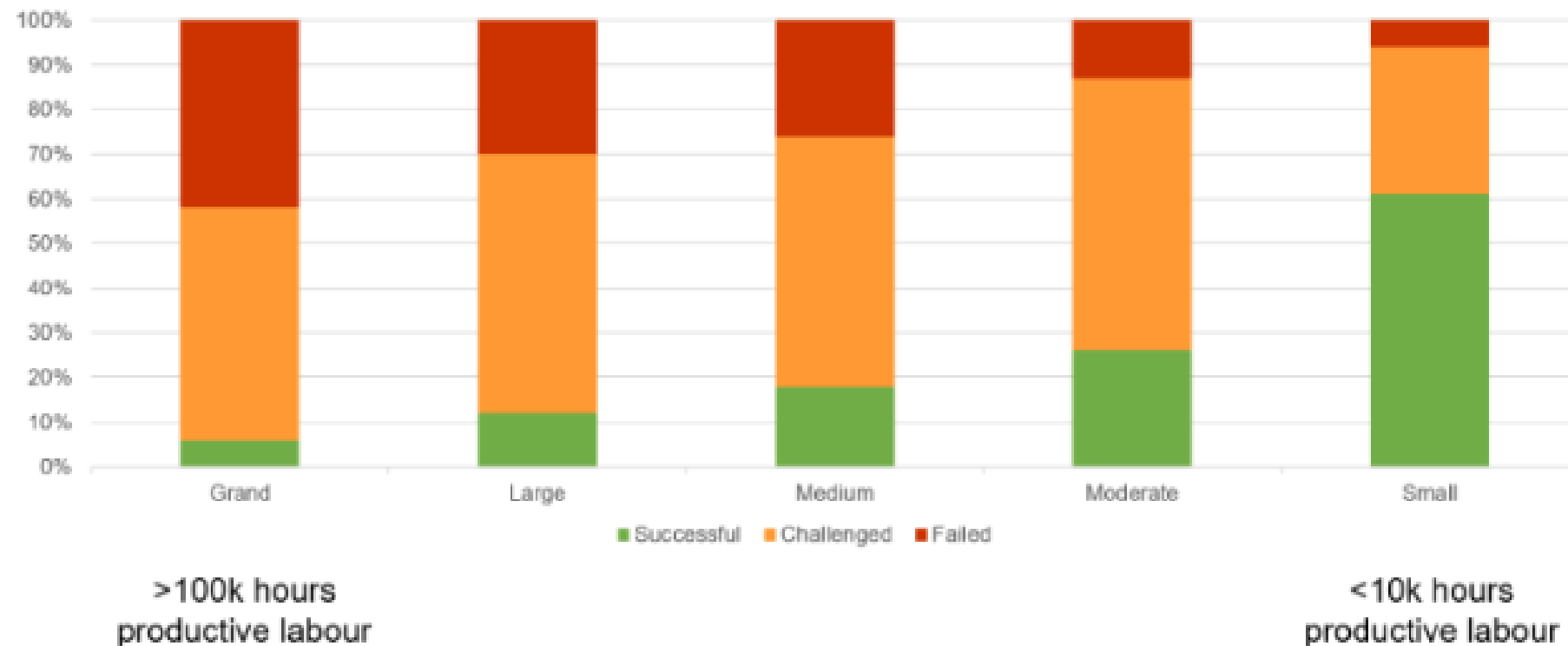
Project Success by Delivery Methodology

Sucessos em projetos de TI



Project Success by Team Maturity

Sucessos em projetos de TI

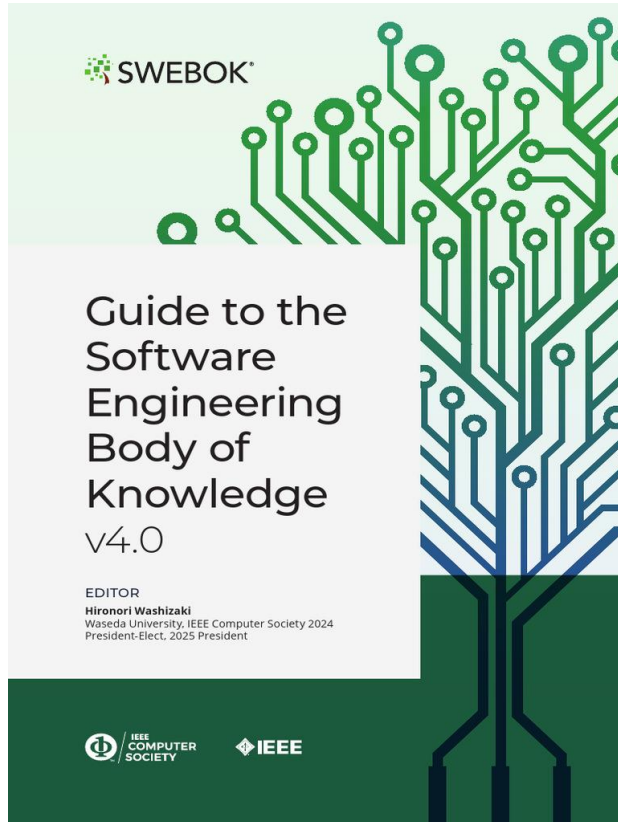


Project Success by Project Size

ES é uma área muito ampla de conhecimentos

- Problemas em não conhecer a área:
 - Gestores de projeto não conseguem formar uma equipe multidisciplinar abrangendo as áreas mais importantes para os projetos
 - Pessoas com funções mais operacionais nos projetos não têm uma visão global da área
 - Não tem noção das possibilidades de aprendizado → crescer profissionalmente
 - Empresas não conseguem avaliar seu portfólio de conhecimento para garantir sua capacidade de entregar projetos de alta qualidade.

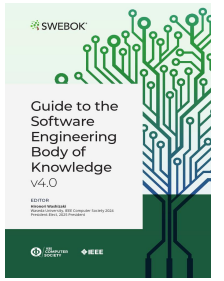
SWEBok



<https://www.computer.org/education/bodies-of-knowledge/software-engineering>

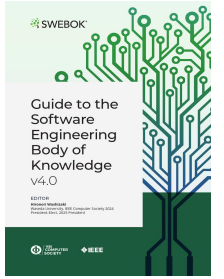
- Toda área profissional se baseia em um corpo de conhecimento
 - Conhecimento comum sobre a teoria envolvida
- *Software Engineering Body of Knowledge* (Guia para o corpo de conhecimento da ES)
 - Alain Abran
 - James Moore
 - Pierre Bourque
 - Robert Depois
- Versão trial 1ª versão em 2001
- 2ª versão em 2004 – 10 áreas aplicada
- 3ª. Versão em 2013 – 15 áreas
- 4ª versão em 2024 – 18 áreas

Swebok



- Iniciativa da IEEE (Instituto de Engenheiros Eletricistas e Eletrônicos)
 - Organização mundial sem fins lucrativos dedicada ao avanço da tecnologia em benefício da humanidade
- Apresenta a ES como um conjunto de áreas de conhecimento necessários para se trabalhar com desenvolvimento de software.
- Desde a codificação em si até a gestão de projetos
- São 18 áreas de conhecimento na V4

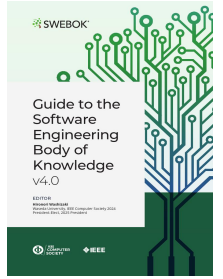
Melhorias na V4



- Agile e DevOps:
 - Essas metodologias foram integradas em quase todas as KAs existentes, deixando de ser tópicos isolados para se tornarem parte do fluxo padrão de engenharia.
- Tecnologias Emergentes:
 - Tópicos como Inteligência Artificial (IA), Machine Learning e Internet das Coisas (IoT) foram incorporados nas áreas de fundamentos e em práticas de desenvolvimento.
- Reestruturação:
 - O guia agora totaliza 18 Áreas de Conhecimento (comparado às 15 da V3), com revisões profundas em áreas como Requisitos de Software para melhor definir as necessidades de stakeholders e requisitos derivados.

SWEBOK - Áreas de conhecimento em ES:

1. Requisitos de software
2. Arquitetura de Software
3. Projeto/Design de software
4. Construção de software
5. Teste de software
6. Operações da ES
7. Manutenção de software
8. Gerencia de configuração de software
9. Gerenciamento de ES
10. Processo de ES
11. Modelos e Métodos de ES
12. Qualidade de software
13. Segurança de Software
14. Práticas profissionais em ES
15. Economia na ES
16. Fundamentos computacionais
17. Fundamentos matemáticos
18. Fundamentos de engenharia



Atividade

- Identificar ao que cada uma das 18 áreas de conhecimento do SWEBook se referem.
- Contendo:
 - Características da área
 - Necessidade e benefícios no contexto de ES
 - Informações relevantes sobre a área
- 1 slide por área – resumir
- Dividir entre grupos de 3 alunos da sala
- Postar nas tarefas do Teams **até o final da aula de hoje!**